US Patent & Trademark Office
Patent Public Search | Text View

United States Patent	12411733
Kind Code	B2
Date of Patent	September 09, 2025
Inventor(s)	Grube; Gary W. et al.

Generating multiple sets of integrity information in a vast storage system

Abstract

A method includes storing a plurality of data in a storage system. A plurality of identifiers corresponding to the plurality of data is determined and the plurality of identifiers are stored in the storage system. A first set of integrity information corresponding to a first system storage level is generated for the plurality of data by performing a first set of cyclic redundancy checks and the first set of integrity information is stored in the storage system. A second set of integrity information corresponding to a second system storage level is generated for the plurality of data and the second set of integrity information is stored in the storage system.

Inventors: **Grube; Gary W. (Barrington Hills, IL), Markison; Timothy W. (Mesa, AZ), Vas; Sebastien (Sunnyvale, CA), Mark; Zachary J. (Chicago, IL), Resch; Jason K. (Warwick, RI)**

Applicant: **Pure Storage, Inc.** (Santa Clara, CA)

Family ID: **81652992**

Assignee: **Pure Storage, Inc. (Santa Clara, CA)**

Appl. No.: **18/363179**

Filed: **August 01, 2023**

Prior Publication Data

Document Identifier	Publication Date
US 20230376380 A1	Nov. 23, 2023

Related U.S. Application Data

continuation parent-doc US 18059833 20221129 US 11755413 child-doc US 18363179
continuation parent-doc US 17743717 20220513 US 11544146 20230103 child-doc US 18059833

continuation parent-doc US 17023971 20200917 US 11080138 20210803 child-doc US 17362251
continuation parent-doc US 14447890 20140731 US 10360180 20190723 child-doc US 16390530
continuation parent-doc US 13154725 20110607 US 10289688 20190514 child-doc US 14447890
continuation-in-part parent-doc US 16137681 20180921 US 10866754 20201215 child-doc US 17023971
continuation-in-part parent-doc US 14454013 20140807 US 10154034 20181211 child-doc US 16137681
continuation-in-part parent-doc US 13021552 20110204 US 9063881 20150623 child-doc US 14454013
continuation-in-part parent-doc US 16390530 20190422 US 11194662 20211207 child-doc US 17362251 20210629
continuation-in-part parent-doc US 12749592 20100330 US 8938591 20150120 child-doc US 14447890 20140731
continuation-in-part parent-doc US 12218594 20080716 US 7962641 20110614 child-doc US 12749592
continuation-in-part parent-doc US 11673613 20071009 US 8285878 20121009 child-doc US 12218594
continuation-in-part parent-doc US 11973622 20071009 US 8171101 20120501 child-doc US 12218594
continuation-in-part parent-doc US 11973542 20071009 US 9996413 20180612 child-doc US 12218594
continuation-in-part parent-doc US 11973621 20071009 US 7904475 20110308 child-doc US 12218594
continuation-in-part parent-doc US 11241555 20050930 US 7953937 20110531 child-doc US 12218594
continuation-in-part parent-doc US 11403684 20060413 US 7574570 20090811 child-doc US 12218594
continuation-in-part parent-doc US 11404071 20060413 US 7574579 20090811 child-doc US 12218594
continuation-in-part parent-doc US 11403391 20060413 US 7546427 20090609 child-doc US 12218594
continuation-in-part parent-doc US 12080042 20080331 US 8880799 20141104 child-doc US 12218594
continuation-in-part parent-doc US 12218200 20080714 US 8209363 20120626 child-doc US 12218594
division parent-doc US 17362251 20210629 US 11340988 20220524 child-doc US 17743717
us-provisional-application US 61327921 20100426
us-provisional-application US 61357430 20100622
us-provisional-application US 61237624 20090827

Publication Classification

Int. Cl.: **G06F11/10** (20060101); **G06F3/06** (20060101)

U.S. Cl.:

CPC **G06F11/1076** (20130101); **G06F3/0619** (20130101); **G06F3/0653** (20130101); **G06F3/067** (20130101); **G06F3/0689** (20130101); **G06F11/1004** (20130101);

Field of Classification Search

CPC: G06F (3/0619); G06F (3/065); G06F (3/0653); G06F (3/067); G06F (3/0689); G06F (11/1004); G06F (11/1076); H03M (13/09)

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
4092732	12/1977	Ouchi	N/A	N/A
4961139	12/1989	Hong	N/A	N/A
5301288	12/1993	Newman	345/543	G06F 12/0292
5404361	12/1994	Casorso	N/A	N/A
5454101	12/1994	Mackay	N/A	N/A
5485474	12/1995	Rabin	N/A	N/A
5488702	12/1995	Byers	N/A	N/A
5632012	12/1996	Belsan	N/A	N/A
5664144	12/1996	Yanai	N/A	N/A
5768623	12/1997	Judd	N/A	N/A
5774643	12/1997	Lubbers	N/A	N/A
5802364	12/1997	Senator	N/A	N/A
5809285	12/1997	Hilland	N/A	N/A
5890156	12/1998	Rekieta	N/A	N/A
5909692	12/1998	Yanai	N/A	N/A
5987622	12/1998	Lo Verso	N/A	N/A
5991414	12/1998	Garay	N/A	N/A
6012159	12/1999	Fischer	N/A	N/A
6052785	12/1999	Lin	N/A	N/A
6058454	12/1999	Gerlach	N/A	N/A
6128277	12/1999	Bruck	N/A	N/A
6151659	12/1999	Solomon	N/A	N/A
6175571	12/2000	Haddock	N/A	N/A
6192472	12/2000	Garay	N/A	N/A
6256688	12/2000	Suetaka	N/A	N/A
6260120	12/2000	Blumenau	N/A	N/A
6272658	12/2000	Steele	N/A	N/A
6301604	12/2000	Nojima	N/A	N/A
6356949	12/2001	Katsandres	N/A	N/A
6366982	12/2001	Suzuki	N/A	N/A
6366995	12/2001	Nikolaevich	N/A	N/A
6374336	12/2001	Peters	N/A	N/A
6415373	12/2001	Peters	N/A	N/A
6418539	12/2001	Walker	N/A	N/A
6449688	12/2001	Peters	N/A	N/A
6567948	12/2002	Steele	N/A	N/A
6571282	12/2002	Bowman-Amuah	N/A	N/A
6606629	12/2002	Dekoning	N/A	N/A
6609223	12/2002	Wolfgang	N/A	N/A

6718361	12/2003	Basani	N/A	N/A
6760808	12/2003	Peters	N/A	N/A
6779003	12/2003	Midgley et al.	N/A	N/A
6785768	12/2003	Peters	N/A	N/A
6785783	12/2003	Buckland	N/A	N/A
6826711	12/2003	Moulton	N/A	N/A
6879596	12/2004	Dooply	N/A	N/A
6964008	12/2004	Van Meter, III	N/A	N/A
7003688	12/2005	Pittelkow	N/A	N/A
7024451	12/2005	Jorgenson	N/A	N/A
7024609	12/2005	Wolfgang	N/A	N/A
7028139	12/2005	Kiselev	N/A	N/A
7080101	12/2005	Watson	N/A	N/A
7103824	12/2005	Halford	N/A	N/A
7103915	12/2005	Redlich	N/A	N/A
7111115	12/2005	Peters	N/A	N/A
7140044	12/2005	Redlich	N/A	N/A
7146644	12/2005	Redlich	N/A	N/A
7149935	12/2005	Morris	N/A	N/A
7171493	12/2006	Shu	N/A	N/A
7222133	12/2006	Raipurkar	N/A	N/A
7240236	12/2006	Cutts	N/A	N/A
7246369	12/2006	Duan	N/A	N/A
7272613	12/2006	Sim	N/A	N/A
7299325	12/2006	Waterhouse	N/A	N/A
7461319	12/2007	Hanam	N/A	N/A
7574570	12/2008	Gladwin et al.	N/A	N/A
7636724	12/2008	De La Torre	N/A	N/A
7657008	12/2009	Zimba	N/A	N/A
7680843	12/2009	Panchbudhe	N/A	N/A
7904475	12/2010	Gladwin	N/A	N/A
7962641	12/2010	Dhuse	N/A	N/A
8082390	12/2010	Fan	N/A	N/A
8171101	12/2011	Gladwin	N/A	N/A
8209363	12/2011	Palthepu	N/A	N/A
8239535	12/2011	Error	709/224	H04L 67/1008
8266237	12/2011	Moore	N/A	N/A
8285878	12/2011	Gladwin	N/A	N/A
8451908	12/2012	MacInnis	375/240.26	H04N 19/176
8464133	12/2012	Grube	N/A	N/A
8626820	12/2013	Levy	N/A	N/A
8938591	12/2014	Mark	N/A	N/A
9229646	12/2015	Todd	N/A	N/A
9332422	12/2015	Bai	N/A	N/A
9401838	12/2015	Brady	N/A	N/A
9411810	12/2015	Mark	N/A	N/A
2002/0062422	12/2001	Butterworth	N/A	N/A
2002/0129230	12/2001	Albright	N/A	N/A
2002/0162075	12/2001	Talagala	N/A	G11B 20/18
2002/0162076	12/2001	Talagala	N/A	N/A

2002/0165913	12/2001	Tokuda	N/A	N/A
2002/0166079	12/2001	Ulrich	N/A	N/A
2002/0170013	12/2001	Bolourchi	N/A	N/A
2003/0018927	12/2002	Gadir	N/A	N/A
2003/0037261	12/2002	Meffert	N/A	N/A
2003/0046587	12/2002	Bheemarasetti	N/A	N/A
2003/0065617	12/2002	Watkins	N/A	N/A
2003/0084020	12/2002	Shu	N/A	N/A
2003/0145270	12/2002	Holt	N/A	N/A
2004/0024963	12/2003	Talagala	N/A	N/A
2004/0122917	12/2003	Menon	N/A	N/A
2004/0123202	12/2003	Talagala	N/A	N/A
2004/0133577	12/2003	Miloushev	N/A	N/A
2004/0133652	12/2003	Miloushev	N/A	N/A
2004/0215998	12/2003	Buxton	N/A	N/A
2004/0228493	12/2003	Ma	N/A	N/A
2004/0243762	12/2003	Brant	N/A	N/A
2005/0080330	12/2004	Masuzawa	N/A	N/A
2005/0086646	12/2004	Zahavi	N/A	N/A
2005/0100022	12/2004	Ramprashad	N/A	N/A
2005/0114594	12/2004	Corbett	N/A	N/A
2005/0125593	12/2004	Karpoff	N/A	N/A
2005/0131993	12/2004	Fatula	N/A	N/A
2005/0132070	12/2004	Redlich	N/A	N/A
2005/0138235	12/2004	Khasid	N/A	N/A
2005/0144382	12/2004	Schmisseur	N/A	N/A
2005/0168460	12/2004	Razdan	N/A	N/A
2005/0229069	12/2004	Hassner	N/A	N/A
2006/0010416	12/2005	Kreck	N/A	N/A
2006/0047907	12/2005	Shiga	N/A	N/A
2006/0136448	12/2005	Cialini	N/A	N/A
2006/0156059	12/2005	Kitamura	N/A	N/A
2006/0224603	12/2005	Correll	N/A	N/A
2006/0224852	12/2005	Kottomtharayil	N/A	N/A
2006/0282744	12/2005	Kounavis	N/A	N/A
2007/0033430	12/2006	Itkis	N/A	N/A
2007/0079081	12/2006	Gladwin	N/A	N/A
2007/0079082	12/2006	Gladwin	N/A	N/A
2007/0079083	12/2006	Gladwin	N/A	N/A
2007/0088970	12/2006	Buxton	N/A	N/A
2007/0101074	12/2006	Patterson	711/156	G06F 16/1744
2007/0174192	12/2006	Gladwin	N/A	N/A
2007/0180272	12/2006	Trezise	N/A	N/A
2007/0180294	12/2006	Kameyama	N/A	N/A
2007/0214285	12/2006	Au	N/A	N/A
2007/0234110	12/2006	Soran	N/A	N/A
2007/0245103	12/2006	Lam	714/E11.123	G06F 11/1451
2007/0283167	12/2006	Venters	N/A	N/A
2008/0183975	12/2007	Foster	N/A	N/A
2008/0282106	12/2007	Shalvi	N/A	N/A

2008/0298470	12/2007	Boyce	N/A	N/A
2009/0006487	12/2008	Gavrilov	N/A	N/A
2009/0037500	12/2008	Kirshenbaum	N/A	N/A
2009/0094250	12/2008	Dhuse	N/A	N/A
2009/0094251	12/2008	Gladwin	N/A	N/A
2009/0094318	12/2008	Gladwin	N/A	N/A
2009/0150631	12/2008	Wilsey	N/A	N/A
2009/0178144	12/2008	Redlich et al.	N/A	N/A
2009/0183056	12/2008	Aston	N/A	N/A
2009/0259667	12/2008	Wang	N/A	N/A
2009/0265278	12/2008	Wang	N/A	N/A
2009/0271454	12/2008	Anglin	N/A	N/A
2009/0313248	12/2008	Balachandran	N/A	N/A
2010/0023524	12/2009	Gladwin	N/A	N/A
2010/0042523	12/2009	Henry	N/A	N/A
2010/0042735	12/2009	Blinn	N/A	N/A
2010/0074149	12/2009	Terada	N/A	N/A
2010/0077447	12/2009	Dholakia	N/A	N/A
2010/0138239	12/2009	Reicher	N/A	N/A
2010/0162076	12/2009	Sim-Tang	N/A	N/A
2010/0268692	12/2009	Resch	N/A	N/A
2011/0029809	12/2010	Dhuse	N/A	N/A
2011/0047192	12/2010	Utsunomiya	N/A	N/A
2011/0107410	12/2010	Dargis	N/A	N/A
2011/0126060	12/2010	Grube	N/A	N/A
2011/0138391	12/2010	Cho	N/A	N/A
2011/0238623	12/2010	Rodriguez	707/626	H04N 21/631
2011/0264989	12/2010	Resch et al.	N/A	N/A
2011/0289122	12/2010	Grube	N/A	N/A

OTHER PUBLICATIONS

Chung; An Automatic Data Segmentation Method for 3D Measured Data Points; National Taiwan University; pp. 1-8; 1998. cited by applicant

Harrison; Lightweight Directory Access Protocol (LDAP): Authentication Methods and Security Mechanisms; IETF Network Working Group; RFC 4513; Jun. 2006; pp. 1-32. cited by applicant

Kubiatowicz, et al.; OceanStore: An Architecture for Global-Scale Persistent Storage; Proceedings of the Ninth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2000); Nov. 2000; pp. 1-12. cited by applicant

LEGG; Lightweight Directory Access Protocol (LDAP): Syntaxes and Matching Rules; IETF Network Working Group; RFC 4517; Jun. 2006; pp. 1-50. cited by applicant

Plank, T1: Erasure Codes for Storage Applications; FAST2005, 4th Usenix Conference on File Storage Technologies; Dec. 13-16, 2005; pp. 1-74. cited by applicant

Rabin; Efficient Dispersal of Information for Security, Load Balancing, and Fault Tolerance; Journal of the Association for Computer Machinery; vol. 36, No. 2; Apr. 1989; pp. 335-348. cited by applicant

Satran, et al.; Internet Small Computer Systems Interface (iSCSI); IETF Network Working Group; RFC 3720; Apr. 2004; pp. 1-257. cited by applicant

Sciberras; Lightweight Directory Access Protocol (LDAP): Schema for User Applications; IETF Network Working Group; RFC 4519; Jun. 2006; pp. 1-33. cited by applicant

Sermersheim; Lightweight Directory Access Protocol (LDAP): The Protocol; IETF Network

Working Group; RFC 4511; Jun. 2006; pp. 1-68. cited by applicant
Shamir; How to Share a Secret; Communications of the ACM; vol. 22, No. 11; Nov. 1979; pp. 612-613. cited by applicant
Smith; Lightweight Directory Access Protocol (LDAP): Uniform Resource Locator; IETF Network Working Group; RFC 4516; Jun. 2006; pp. 1-15. cited by applicant
Smith; Lightweight Directory Access Protocol (LDAP): String Representation of Search Filters; IETF Network Working Group; RFC 4515; Jun. 2006; pp. 1-12. cited by applicant
Wildi; Java iSCSi Initiator; Master Thesis; Department of Computer and Information Science, University of Konstanz; Feb. 2007; 60 pgs. cited by applicant
Xin, et al.; Evaluation of Distributed Recovery in Large-Scale Storage Systems; 13th IEEE International Symposium on High Performance Distributed Computing; Jun. 2004; pp. 172-181. cited by applicant
Zeilenga; Lightweight Directory Access Protocol (LDAP): Directory Information Models; IETF Network Working Group; RFC 4512; Jun. 2006; pp. 1-49. cited by applicant
Zeilenga; Lightweight Directory Access Protocol (LDAP): Internationalized String Preparation; IETF Network Working Group; RFC 4518; Jun. 2006; pp. 1-14. cited by applicant
Zeilenga; Lightweight Directory Access Protocol (LDAP): String Representation of Distinguished Names; IETF Network Working Group; RFC 4514; Jun. 2006; pp. 1-15. cited by applicant
Zeilenga; Lightweight Directory Access Protocol (LDAP): Technical Specification Road Map; IETF Network Working Group; RFC 4510; Jun. 2006; pp. 1-8. cited by applicant

---

Primary Examiner: Decady; Albert

Assistant Examiner: Kabir; Enamul M

Attorney, Agent or Firm: Katz Ruby & Carle LLP

Background/Summary

CROSS-REFERENCE TO RELATED APPLICATIONS (1) The present U.S. Utility patent application claims priority pursuant to 35 U.S.C. § 120 as a continuation of U.S. Utility application Ser. No. 18/059,833, entitled “UTILIZING INTEGRITY INFORMATION TO DETERMINE CORRUPTION IN A VAST STORAGE SYSTEM”, filed Nov. 29, 2022, which is a continuation of U.S. Utility application Ser. No. 17/743,717, entitled “UTILIZING INTEGRITY INFORMATION IN A VAST STORAGE SYSTEM”, filed May 13, 2022, issued as U.S. Pat. No. 11,544,146 on Jan. 3, 2023, which claims priority pursuant to 35 U.S.C. § 121 as a divisional of U.S. Utility application Ser. No. 17/362,251, entitled “GENERATING INTEGRITY INFORMATION IN A VAST STORAGE SYSTEM”, filed Jun. 29, 2021, issued as U.S. Pat. No. 11,340,988 on May 24, 2022, which is a continuation of U.S. Utility application Ser. No. 17/023,971, entitled “STORING INTEGRITY INFORMATION IN A VAST STORAGE SYSTEM”, filed Sep. 17, 2020, issued as U.S. Pat. No. 11,080,138 on Aug. 3, 2021, which is a continuation-in-part (CIP) of U.S. Utility application Ser. No. 16/137,681, entitled “CONTENT ARCHIVING IN A DISTRIBUTED STORAGE NETWORK”, filed Sep. 21, 2018, issued as U.S. Pat. No. 10,866,754 on Dec. 15, 2020, which is a continuation-in-part (CIP) of U.S. Utility application Ser. No. 14/454,013, entitled “COOPERATIVE DATA ACCESS REQUEST AUTHORIZATION IN A DISPERSED STORAGE NETWORK”, filed Aug. 7, 2014, issued as U.S. Pat. No. 10,154,034 on Dec. 11, 2018, which is a continuation-in-part (CIP) of U.S. Utility application Ser. No. 13/021,552, entitled “SLICE RETRIEVAL IN ACCORDANCE WITH AN ACCESS SEQUENCE IN A DISPERSED STORAGE NETWORK”, filed Feb. 4, 2011, issued as U.S. Pat. No. 9,063,881 on Jun. 23, 2015,

which claims priority pursuant to 35 U.S.C. § 119(e) to U.S. Provisional Application No. 61/327,921, entitled “SYSTEM ACCESS AND DATA INTEGRITY VERIFICATION IN A DISPERSED STORAGE SYSTEM”, filed Apr. 26, 2010, all of which are hereby incorporated herein by reference in their entirety and made part of the present U.S. Utility patent application for all purposes. (2) U.S. Utility application Ser. No. 17/362,251 is also a continuation-in-part of U.S. Utility application Ser. No. 16/390,530, entitled “DIGEST LISTING DECOMPOSITION”, filed Apr. 22, 2019, issued as U.S. Pat. No. 11,194,662 on Dec. 7, 2021, which is a continuation of U.S. Utility application Ser. No. 14/447,890, entitled “DIGEST LISTING DECOMPOSITION”, filed Jul. 31, 2014, issued as U.S. Pat. No. 10,360,180 on Jul. 23, 2019, which is a continuation of U.S. Utility application Ser. No. 13/154,725, entitled, “METADATA ACCESS IN A DISPERSED STORAGE NETWORK”, filed Jun. 7, 2011, issued as U.S. Pat. No. 10,289,688 on May 14, 2019, which claims priority pursuant to 35 U.S.C. § 119(e) to U.S. Provisional Application No. 61/357,430, entitled “DISPERSAL METHOD IN A DISPERSED STORAGE SYSTEM”, filed Jun. 22, 2010, all of which are hereby incorporated herein by reference in their entirety and made part of the present U.S. Utility patent application for all purposes. (3) U.S. Utility application Ser. No. 14/447,890 also claims priority pursuant to 35 U.S.C. § 120 as a continuation-in-part of U.S. Utility application Ser. No. 12/749,592, entitled “DISPERSED STORAGE PROCESSING UNIT AND METHODS WITH DATA AGGREGATION FOR USE IN A DISPERSED STORAGE SYSTEM”, filed Mar. 30, 2010, issued as U.S. Pat. No. 8,938,591 on Jan. 20, 2015, which claims priority pursuant to 35 U.S.C. § 119(e) to U.S. Provisional Application No. 61/237,624, entitled “DISPERSED STORAGE UNIT AND METHODS WITH METADATA SEPARATION FOR USE IN A DISPERSED STORAGE SYSTEM”, filed Aug. 27, 2009, all of which are hereby incorporated herein by reference in their entirety and made part of the present U.S. Utility patent application for all purposes. (4) U.S. Utility application Ser. No. 12/749,592 also claims priority pursuant to 35 U.S.C. § 120 as a continuation-in-part of U.S. Utility application Ser. No. 12/218,594, entitled STREAMING MEDIA SOFTWARE INTERFACE TO A DISPERSED DATA STORAGE NETWORK, filed Jul. 16, 2008, issued as U.S. Pat. No. 7,962,641 on Jun. 14, 2011, which claims priority pursuant to 35 U.S.C. § 120 as a continuation-in-part of the following, all of which are hereby incorporated herein by reference in their entirety and made part of the present U.S. Utility patent application for all purposes: 1. U.S. Utility application Ser. No. 11/973,613, entitled BLOCK BASED ACCESS TO A DISPERSED DATA STORAGE NETWORK, filed Oct. 9, 2007, issued as U.S. Pat. No. 8,285,878 on Oct. 9, 2012; 2. U.S. Utility application Ser. No. 11/973,622, entitled SMART ACCESS TO A DISPERSED DATA STORAGE NETWORK, filed Oct. 9, 2007, issued as U.S. Pat. No. 8,171,101 on May 1, 2012; 3. U.S. Utility application Ser. No. 11/973,542, entitled ENSURING DATA INTEGRITY ON A DISPERSED STORAGE NETWORK, filed Oct. 9, 2007, issued as U.S. Pat. No. 9,996,413 on Jun. 12, 2018; 4. U.S. Utility application Ser. No. 11/973,621, entitled VIRTUALIZED STORAGE VAULTS ON A DISPERSED DATA STORAGE NETWORK, filed Oct. 9, 2007, issued as U.S. Pat. No. 7,904,475 on Mar. 8, 2011; 5. U.S. Utility application Ser. No. 11/241,555, entitled SYSTEM, METHODS, AND APPARATUS FOR SUBDIVIDING DATA FOR STORAGE IN A DISPERSED DATA STORAGE GRID, filed Sep. 30, 2005, which issued as U.S. Pat. No. 7,953,937 on May 31, 2011; 6. U.S. Utility application Ser. No. 11/403,684, entitled BILLING SYSTEM FOR INFORMATION DISPERSAL SYSTEM, filed Apr. 13, 2006, issued as U.S. Pat. No. 7,574,570 on Aug. 11, 2009; 7. U.S. Utility application Ser. No. 11/404,071, entitled METADATA MANAGEMENT SYSTEM FOR AN INFORMATION DISPERSED STORAGE SYSTEM, filed Apr. 13, 2006, issued as U.S. Pat. No. 7,574,579 on Aug. 11, 2009; 8. U.S. Utility application Ser. No. 11/403,391, entitled SYSTEM FOR REBUILDING DISPERSED DATA, filed Apr. 13, 2006, issued as U.S. Pat. No. 7,546,427 on Jun. 9, 2009; 9. U.S. Utility application Ser. No. 12/080,042, entitled REBUILDING DATA ON A DISPERSED STORAGE NETWORK, filed Mar. 31, 2008; issued as U.S. Pat. No. 8,880,799 on Nov. 4, 2014, and 10. U.S. Utility application Ser. No. 12/218,200, entitled FILE

SYSTEM ADAPTED FOR USE WITH A DISPERSED DATA STORAGE NETWORK, filed Jul. 14, 2008, issued as U.S. Pat. No. 8,209,363 on Jun. 26, 2012. (5) In addition, U.S. Utility application Ser. No. 16/390,530, is related to the following U.S. patent applications that are commonly owned, all of which are hereby incorporated herein by reference in their entirety and made part of the present U.S. Utility patent application for all purposes: 1. DISPERSED STORAGE UNIT AND METHODS WITH METADATA SEPARATION FOR USE IN A DISPERSED STORAGE SYSTEM, Ser. No. 12/749,583, and filed on Mar. 30, 2010, issued as U.S. Pat. No. 9,235,350 on Jan. 12, 2016. 2. DISPERSED STORAGE PROCESSING UNIT AND METHODS WITH OPERATING SYSTEM DIVERSITY FOR USE IN A DISPERSED STORAGE SYSTEM, Ser. No. 12/749,606, and filed on Mar. 30, 2010, issued as U.S. Pat. No. 9,690,513 on Jun. 27, 2017. 3. DISPERSED STORAGE PROCESSING UNIT AND METHODS WITH GEOGRAPHICAL DIVERSITY FOR USE IN A DISPERSED STORAGE SYSTEM, Ser. No. 12/749,625, and filed on Mar. 3, 2011, issued as U.S. Pat. No. 9,772,791 on Sep. 26, 2017.

BACKGROUND

Technical Field

(1) This invention relates generally to computer networks and more particularly to dispersing error encoded data.

Description of Related Art

(2) Computing devices are known to communicate data, process data, and/or store data. Such computing devices range from wireless smart phones, laptops, tablets, personal computers (PC), work stations, and video game devices, to data centers that support millions of web searches, stock trades, or on-line purchases every day. In general, a computing device includes a central processing unit (CPU), a memory system, user input/output interfaces, peripheral device interfaces, and an interconnecting bus structure.

(3) As is further known, a computer may effectively extend its CPU by using “cloud computing” to perform one or more computing functions (e.g., a service, an application, an algorithm, an arithmetic logic function, etc.) on behalf of the computer. Further, for large services, applications, and/or functions, cloud computing may be performed by multiple cloud computing resources in a distributed manner to improve the response time for completion of the service, application, and/or function. For example, Hadoop is an open source software framework that supports distributed applications enabling application execution by thousands of computers.

(4) In addition to cloud computing, a computer may use “cloud storage” as part of its memory system. As is known, cloud storage enables a user, via its computer, to store files, applications, etc. on an Internet storage system. The Internet storage system may include a RAID (redundant array of independent disks) system and/or a dispersed storage system that uses an error correction scheme to encode data for storage.

(5) Various conventional storage systems are used to archive user data. Usually, however, the data to be archived requires a user to specify a file path to the data to be stored in an archive, or by requiring a user to specify particular file or object name for storage.

Description

BRIEF DESCRIPTION OF THE DRAWINGS

(1) FIG. **1** is a schematic block diagram of an embodiment of a dispersed or distributed storage network (DSN) in accordance with the present invention;

(2) FIG. **2** is a schematic block diagram of an embodiment of a computing core in accordance with the present invention;

(3) FIG. **3** is a schematic block diagram of an example of dispersed storage error encoding of data in accordance with the present invention;

(4) FIG. **4** is a schematic block diagram of a generic example of an error encoding function in accordance with the present invention;
(5) FIG. **5** is a schematic block diagram of a specific example of an error encoding function in accordance with the present invention;
(6) FIG. **6** is a schematic block diagram of an example of a slice name of an encoded data slice (EDS) in accordance with the present invention;
(7) FIG. **7** is a schematic block diagram of an example of dispersed storage error decoding of data in accordance with the present invention;
(8) FIG. **8** is a schematic block diagram of a generic example of an error decoding function in accordance with the present invention;
(9) FIG. **9** is a schematic block diagram of another embodiment of a computing system in accordance with the present invention;
(10) FIG. **10** is a flowchart illustrating an example of archiving data in accordance with the present invention.
(11) FIG. **11** is a flowchart illustrating an example of generating integrity information in accordance with the invention; and
(12) FIG. **12** is a flowchart illustrating an example of verifying slice integrity in accordance with the invention.

DETAILED DESCRIPTION

(13) FIG. **1** is a schematic block diagram of an embodiment of a dispersed, or distributed, storage network (DSN) **10** that includes a plurality of computing devices **12-16**, a managing unit **18**, an integrity processing unit **20**, and a DSN memory **22**. The components of the DSN **10** are coupled to a network **24**, which may include one or more wireless and/or wire lined communication systems; one or more non-public intranet systems and/or public internet systems; and/or one or more local area networks (LAN) and/or wide area networks (WAN).
(14) The DSN memory **22** includes a plurality of storage units **36** that may be located at geographically different sites (e.g., one in Chicago, one in Milwaukee, etc.), at a common site, or a combination thereof. For example, if the DSN memory **22** includes eight storage units **36**, each storage unit is located at a different site. As another example, if the DSN memory **22** includes eight storage units **36**, all eight storage units are located at the same site. As yet another example, if the DSN memory **22** includes eight storage units **36**, a first pair of storage units are at a first common site, a second pair of storage units are at a second common site, a third pair of storage units are at a third common site, and a fourth pair of storage units are at a fourth common site. Note that a DSN memory **22** may include more or less than eight storage units **36**. Further note that each storage unit **36** includes a computing core (as shown in FIG. **2**, or components thereof) and a plurality of memory devices for storing dispersed error encoded data.
(15) Each of the computing devices **12-16**, the managing unit **18**, and the integrity processing unit **20** include a computing core **26**, which includes network interfaces **30-33**. Computing devices **12-16** may each be a portable computing device and/or a fixed computing device. A portable computing device may be a social networking device, a gaming device, a cell phone, a smart phone, a digital assistant, a digital music player, a digital video player, a laptop computer, a handheld computer, a tablet, a video game controller, and/or any other portable device that includes a computing core. A fixed computing device may be a computer (PC), a computer server, a cable set-top box, a satellite receiver, a television set, a printer, a fax machine, home entertainment equipment, a video game console, and/or any type of home or office computing equipment. Note that each of the managing unit **18** and the integrity processing unit **20** may be separate computing devices, may be a common computing device, and/or may be integrated into one or more of the computing devices **12-16** and/or into one or more of the storage units **36**.
(16) Each interface **30**, **32**, and **33** includes software and hardware to support one or more communication links via the network **24** indirectly and/or directly. For example, interface **30**

supports a communication link (e.g., wired, wireless, direct, via a LAN, via the network **24**, etc.) between computing devices **14** and **16**. As another example, interface **32** supports communication links (e.g., a wired connection, a wireless connection, a LAN connection, and/or any other type of connection to/from the network **24**) between computing devices **12** and **16** and the DSN memory **22**. As yet another example, interface **33** supports a communication link for each of the managing unit **18** and the integrity processing unit **20** to the network **24**.

(17) Computing devices **12** and **16** include a dispersed storage (DS) client module **34**, which enables the computing device to dispersed storage error encode and decode data (e.g., data **40**) as subsequently described with reference to one or more of FIGS. **3-8**. In this example embodiment, computing device **16** functions as a dispersed storage processing agent for computing device **14**. In this role, computing device **16** dispersed storage error encodes and decodes data on behalf of computing device **14**. With the use of dispersed storage error encoding and decoding, the DSN **10** is tolerant of a significant number of storage unit failures (the number of failures is based on parameters of the dispersed storage error encoding function) without loss of data and without the need for a redundant or backup copies of the data. Further, the DSN **10** stores data for an indefinite period of time without data loss and in a secure manner (e.g., the system is very resistant to unauthorized attempts at accessing the data).

(18) In operation, the managing unit **18** performs DS management services. For example, the managing unit **18** establishes distributed data storage parameters (e.g., vault creation, distributed storage parameters, security parameters, billing information, user profile information, etc.) for computing devices **12-14** individually or as part of a group of user devices. As a specific example, the managing unit **18** coordinates creation of a vault (e.g., a virtual memory block associated with a portion of an overall namespace of the DSN) within the DSN memory **22** for a user device, a group of devices, or for public access and establishes per vault dispersed storage (DS) error encoding parameters for a vault. The managing unit **18** facilitates storage of DS error encoding parameters for each vault by updating registry information of the DSN **10**, where the registry information may be stored in the DSN memory **22**, a computing device **12-16**, the managing unit **18**, and/or the integrity processing unit **20**.

(19) The managing unit **18** creates and stores user profile information (e.g., an access control list (ACL)) in local memory and/or within memory of the DSN memory **22**. The user profile information includes authentication information, permissions, and/or the security parameters. The security parameters may include encryption/decryption scheme, one or more encryption keys, key generation scheme, and/or data encoding/decoding scheme.

(20) The managing unit **18** creates billing information for a particular user, a user group, a vault access, public vault access, etc. For instance, the managing unit **18** tracks the number of times a user accesses a non-public vault and/or public vaults, which can be used to generate a per-access billing information. In another instance, the managing unit **18** tracks the amount of data stored and/or retrieved by a user device and/or a user group, which can be used to generate a per-data-amount billing information.

(21) As another example, the managing unit **18** performs network operations, network administration, and/or network maintenance. Network operations includes authenticating user data allocation requests (e.g., read and/or write requests), managing creation of vaults, establishing authentication credentials for user devices, adding/deleting components (e.g., user devices, storage units, and/or computing devices with a DS client module **34**) to/from the DSN **10**, and/or establishing authentication credentials for the storage units **36**. Network administration includes monitoring devices and/or units for failures, maintaining vault information, determining device and/or unit activation status, determining device and/or unit loading, and/or determining any other system level operation that affects the performance level of the DSN **10**. Network maintenance includes facilitating replacing, upgrading, repairing, and/or expanding a device and/or unit of the DSN **10**.

(22) The integrity processing unit **20** performs rebuilding of 'bad' or missing encoded data slices. At a high level, the integrity processing unit **20** performs rebuilding by periodically attempting to retrieve/list encoded data slices, and/or slice names of the encoded data slices, from the DSN memory **22**. For retrieved encoded slices, they are checked for errors due to data corruption, outdated version, etc. If a slice includes an error, it is flagged as a 'bad' slice. For encoded data slices that were not received and/or not listed, they are flagged as missing slices. Bad and/or missing slices are subsequently rebuilt using other retrieved encoded data slices that are deemed to be good slices to produce rebuilt slices. The rebuilt slices are stored in the DSN memory **22**.
(23) FIG. **2** is a schematic block diagram of an embodiment of a computing core **26** that includes a processing module **50**, a memory controller **52**, main memory **54**, a video graphics processing unit **55**, an input/output (IO) controller **56**, a peripheral component interconnect (PCI) interface **58**, an IO interface module **60**, at least one IO device interface module **62**, a read only memory (ROM) basic input output system (BIOS) **64**, and one or more memory interface modules. The one or more memory interface module(s) includes one or more of a universal serial bus (USB) interface module **66**, a host bus adapter (HBA) interface module **68**, a network interface module **70**, a flash interface module **72**, a hard drive interface module **74**, and a DSN interface module **76**.
(24) The DSN interface module **76** functions to mimic a conventional operating system (OS) file system interface (e.g., network file system (NFS), flash file system (FFS), disk file system (DFS), file transfer protocol (FTP), web-based distributed authoring and versioning (WebDAV), etc.) and/or a block memory interface (e.g., small computer system interface (SCSI), internet small computer system interface (iSCSI), etc.). The DSN interface module **76** and/or the network interface module **70** may function as one or more of the interface **30**-**33** of FIG. **1**. Note that the IO device interface module **62** and/or the memory interface modules **66**-**76** may be collectively or individually referred to as IO ports.
(25) FIG. **3** is a schematic block diagram of an example of dispersed storage error encoding of data. When a computing device **12** or **16** has data to store it disperse storage error encodes the data in accordance with a dispersed storage error encoding process based on dispersed storage error encoding parameters. The dispersed storage error encoding parameters include an encoding function (e.g., information dispersal algorithm, Reed-Solomon, Cauchy Reed-Solomon, systematic encoding, non-systematic encoding, on-line codes, etc.), a data segmenting protocol (e.g., data segment size, fixed, variable, etc.), and per data segment encoding values. The per data segment encoding values include a total, or pillar width, number (T) of encoded data slices per encoding of a data segment (i.e., in a set of encoded data slices); a decode threshold number (D) of encoded data slices of a set of encoded data slices that are needed to recover the data segment; a read threshold number (R) of encoded data slices to indicate a number of encoded data slices per set to be read from storage for decoding of the data segment; and/or a write threshold number (W) to indicate a number of encoded data slices per set that must be accurately stored before the encoded data segment is deemed to have been properly stored. The dispersed storage error encoding parameters may further include slicing information (e.g., the number of encoded data slices that will be created for each data segment) and/or slice security information (e.g., per encoded data slice encryption, compression, integrity checksum, etc.).
(26) In the present example, Cauchy Reed-Solomon has been selected as the encoding function (a generic example is shown in FIG. **4** and a specific example is shown in FIG. **5**); the data segmenting protocol is to divide the data object into fixed sized data segments; and the per data segment encoding values include: a pillar width of 5, a decode threshold of 3, a read threshold of 4, and a write threshold of 4. In accordance with the data segmenting protocol, the computing device **12** or **16** divides the data (e.g., a file (e.g., text, video, audio, etc.), a data object, or other data arrangement) into a plurality of fixed sized data segments (e.g., **1** through Y of a fixed size in range of Kilo-bytes to Tera-bytes or more). The number of data segments created is dependent of the size of the data and the data segmenting protocol.

(27) The computing device **12** or **16** then disperse storage error encodes a data segment using the selected encoding function (e.g., Cauchy Reed-Solomon) to produce a set of encoded data slices. FIG. **4** illustrates a generic Cauchy Reed-Solomon encoding function, which includes an encoding matrix (EM), a data matrix (DM), and a coded matrix (CM). The size of the encoding matrix (EM) is dependent on the pillar width number (T) and the decode threshold number (D) of selected per data segment encoding values. To produce the data matrix (DM), the data segment is divided into a plurality of data blocks and the data blocks are arranged into D number of rows with Z data blocks per row. Note that Z is a function of the number of data blocks created from the data segment and the decode threshold number (D). The coded matrix is produced by matrix multiplying the data matrix by the encoding matrix.

(28) FIG. **5** illustrates a specific example of Cauchy Reed-Solomon encoding with a pillar number (T) of five and decode threshold number of three. In this example, a first data segment is divided into twelve data blocks (D**1**-D**12**). The coded matrix includes five rows of coded data blocks, where the first row of X**11**-X**14** corresponds to a first encoded data slice (EDS **1_1**), the second row of X**21**-X**24** corresponds to a second encoded data slice (EDS **2_1**), the third row of X**31**-X**34** corresponds to a third encoded data slice (EDS **3_1**), the fourth row of X**41**-X**44** corresponds to a fourth encoded data slice (EDS **4_1**), and the fifth row of X**51**-X**54** corresponds to a fifth encoded data slice (EDS **5_1**). Note that the second number of the EDS designation corresponds to the data segment number.

(29) Returning to the discussion of FIG. **3**, the computing device also creates a slice name (SN) for each encoded data slice (EDS) in the set of encoded data slices. A typical format for a slice name **80** is shown in FIG. **6**. As shown, the slice name (SN) **80** includes a pillar number of the encoded data slice (e.g., one of **1**-T), a data segment number (e.g., one of **1**-Y), a vault identifier (ID), a data object identifier (ID), and may further include revision level information of the encoded data slices. The slice name functions as, at least part of, a DSN address for the encoded data slice for storage and retrieval from the DSN memory **22**.

(30) As a result of encoding, the computing device **12** or **16** produces a plurality of sets of encoded data slices, which are provided with their respective slice names to the storage units for storage. As shown, the first set of encoded data slices includes EDS **1_1** through EDS **5_1** and the first set of slice names includes SN **1_1** through SN **5_1** and the last set of encoded data slices includes EDS **1**_Y through EDS **5**_Y and the last set of slice names includes SN **1**_Y through SN **5**_Y.

(31) FIG. **7** is a schematic block diagram of an example of dispersed storage error decoding of a data object that was dispersed storage error encoded and stored in the example of FIG. **4**. In this example, the computing device **12** or **16** retrieves from the storage units at least the decode threshold number of encoded data slices per data segment. As a specific example, the computing device retrieves a read threshold number of encoded data slices.

(32) To recover a data segment from a decode threshold number of encoded data slices, the computing device uses a decoding function as shown in FIG. **8**. As shown, the decoding function is essentially an inverse of the encoding function of FIG. **4**. The coded matrix includes a decode threshold number of rows (e.g., three in this example) and the decoding matrix in an inversion of the encoding matrix that includes the corresponding rows of the coded matrix. For example, if the coded matrix includes rows **1**, **2**, and **4**, the encoding matrix is reduced to rows **1**, **2**, and **4**, and then inverted to produce the decoding matrix.

(33) FIGS. **9** and **10** illustrate particular embodiments in which content data stored in a user device, or data transmitted between a user device and an external device, can be automatically and conditionally archived using a distributed storage network (DSN). For example, a DS processing agent inside of a device (e.g., a smart phone, a land based phone, a laptop, desktop, the cable box, a home security system, a home automation system, etc.) grabs content, filters it, sorts it, and stores it in a DSN memory. For example: banking info, home video, pictures, e-mail, SMS, class notes, website visits, contacts, connections, grades, medical records, social networking messaging, and/or

password lists. The DS processing agent correlates the data to preferences to determine how much content to save, and how often to store new content. The agent also determines operational parameters associated with the DSN based on one or more of the data type, age, priority, status, etc. In some implementations, the DS processing utilizes two different DS units to store different types of critical information, or to store particular types of critical information in pillars associated with two different DS units.

(34) FIG. **9** is a schematic block diagram of another embodiment of a computing system that includes a user device domain **272**, a dispersed storage (DS) processing unit **96**, such as computing device **16**, and a dispersed storage network (DSN) memory **22**. The user device domain **272** includes user devices **201**-**203**. Note that the user device domain **272** may include any number of user devices. The DS processing unit **96** includes a DS processing module **94** and the DSN memory **22** includes a plurality of 1.sup.st-N.sup.th DS units. Such user devices **201**-**203** of the user device domain **272** are associated with a common user such that data, information, and/or messages traversed by the user devices **201**-**203** share relationship with the common user. The DS processing unit **96** provides user device **201** access to the DSN memory **22** when the user device **201** does not include a DS processing module **94**, such as DS client module **34**.

(35) The user devices **201**-**203** may include fixed or portable devices as discussed previously (e.g., a smart phone, a wired phone, a laptop computer, a tablet computer, a desktop computer, a cable set-top box, a smart appliance, a home security system, a home automation system, etc.). The user devices **201**-**203** may include a computing core, one or more interfaces, the DS processing module **94** and/or a collection module **274**. For example, user device **201** includes the collection module **274**. User device **202** includes the collection module **274** and the DS processing module **94**. User device **3** includes the DS processing module **94** which includes the collection module **274**. The collection module **274** includes a functional entity (e.g., a software application that runs on a computing core or as part of a processing module) that intercepts user data, processes the user data to produce a data representation, and/or facilitates storage of the data representation in the DSN memory in accordance with one or more of metadata, preferences, and/or operational parameters (e.g., dispersed storage error coding parameters).

(36) In an example of operation, the user devices **201**-**203** traverse the user data from time to time where the user data may include one or more of banking information, home video, video broadcasts, pictures from a user camera, e-mail messages, short message service messages, class notes, website visits, web downloads, contact lists, social networking connections, school grades, medical records, social networking messaging, password lists, and any other user data type associated with the user. Note that the user data may be communicated from one user device to another user device and/or from a user device to a module or unit external to the computing system. Further note that the user data may be stored in any one or more of the user devices **201**-**203**.

(37) In another example of operation, the collection module **274** of user device **201** intercepts medical records that are being processed by user device **201**. The collection module **274** determines metadata based on the medical records and determines preferences based on a user identifier (ID). The collection module **274** determines whether to archive the medical records based in part on the medical records, the metadata, and the preferences. The collection module **274** processes the medical records in accordance with the preferences to produce a data representation when the collection module **274** determines to archive the medical records. For example, the collection module **274** of the user device **201** sends the data representation **275** to the DS processing unit **96**. The data representation **275** may include one or more of the data, the metadata, the preferences, and storage guidance. The DS processing unit **96** determines operational parameters, creates encoded data slices based on the data representation, and sends the encoded data slices **11** to the DSN memory **22** with a store command to store the encoded data slices **11**. As another example, the collection module **274** of the user device **201** determines operational parameters based in part on one or more of the user data, the metadata, the preferences, and the data representation. Next,

the collection module **274** sends the data representation **275** to the DS processing unit **96**. In this example, the data representation **275** may include one or more of the operational parameters, the metadata, the preferences, and storage guidance. The DS processing unit **96** determines final operational parameters based in part on the operational parameters from the collection module **274**, creates encoded data slices based on the data representation and the final operational parameters, and sends the encoded data slices **11** to the DSN memory **22** with a store command to store the encoded data slices **11**.

(38) In yet another example of operation, the collection module **274** of user device **202** intercepts banking records that are being viewed by user device **202**. The collection module **274** determines metadata based on the banking records and determines preferences based on a user ID. The collection module **274** determines whether to archive the banking records based on the banking records, the metadata, and the preferences. The collection module **274** processes the banking records in accordance with the preferences to produce a data representation when the collection module determines to archive the banking records. For example, the collection module **274** sends the data representation to the DS processing module **94** of the 2.sup.nd DS such that the data representation may include one or more of the metadata, the preferences, and storage guidance. The DS processing module **94** determines operational parameters, creates encoded data slices based on the data representation, and sends the encoded data slices **11** to the DSN memory **22** with a store command to store the encoded data slices **11**. As another example, the collection module **274** determines operational parameters based on one or more of the user data (e.g., the banking records), the metadata, the preferences, and the data representation. The collection module **274** sends the data representation to the DS processing module **94** of the 2.sup.nd DS unit, wherein the data representation includes one or more of the operational parameters, the metadata, the preferences, and storage guidance. The DS processing module **94** determines final operational parameters based in part on the operational parameters from the collection module, creates encoded data slices based on the data representation and the final operational parameters, and sends the encoded data slices **11** to the DSN memory **22** with a store command to store the encoded data slices **11**.

(39) In a further example of operation, the collection module **274** of user device **203** intercepts home video files that are being processed by user device **203**. The collection module **274** determines metadata based on one or more of the home video files and determines preferences based in part on a user ID. The collection module **274** determines whether to archive the home video files based on the home video files, the metadata, and the preferences. The collection module **274** processes the home video files in accordance with the preferences to produce a data representation when the collection module **274** determines to archive the home video files. For example, the collection module **274** sends the data representation to the DS processing module **94** of the 3.sup.rd DS unit, wherein the data representation includes one or more of the metadata, the preferences, and storage guidance. The DS processing module **94** determines operational parameters, creates encoded data slices based on the data representation and the operational parameters, and sends the encoded data slices **11** to the DSN memory **22** with a store command to store the encoded data slices **11**. As another example, the collection module **274** determines operational parameters based on one or more of the user data (e.g., the home video files), the metadata, the preferences, and the data representation. The collection module **274** sends the data representation to the DS processing module **94** of the 3.sup.rd DS unit, wherein the data representation includes one or more of the operational parameters, the metadata, the preferences, and storage guidance. The DS processing module **94** determines final operational parameters based on the operational parameters from the collection module **274**, creates encoded data slices based on the data representation and the final operational parameters, and sends the encoded data slices **11** to the DSN memory **22** with a store command to store the encoded data slices **11**.

(40) FIG. **10** is a flowchart illustrating an example of archiving data. The method begins with step

276 where the processing module captures user data. Such capturing may include one or more of monitoring a data stream between a user device and an external entity, monitoring a data stream internally between functional elements within the user device, and retrieving stored data from a memory of the user device. The method continues at step **278** where the processing module determines metadata, wherein the metadata may include one or more of a user identifier (ID), a data type, a source indicator, a destination indicator, a context indicator, a priority indicator, a status indicator, a time indicator, and a date indicator. Such a determination may be based on one or more of the captured user data, current activity or activities of the user device (e.g., active processes, machines state, input/output utilization, memory utilization, etc.), geographic location information, clock information, a sensor input, a user record, a lookup, a command, a predetermination, and message. For example, the processing module determines the metadata to include a banking record data type indicator and a geographic location-based context indicator when the processing module determines the banking data type and geographic location information.

(41) The method continues with step **280** where the processing module determines preferences, wherein the preferences may include one or more of archiving priority by data type, archiving frequency, context priority, status priority, volume priority, performance requirements, and reliability requirements. Such a determination may be based on one or more of the user ID, the user data, the metadata, context information, a lookup, a predetermination, a command, a query response, and a message. The method continues at step **282** where the processing module determines whether to archive data based on one or more of the metadata, context information, a user ID, a lookup, the preferences, and a comparison of the metadata to one or more thresholds. For example, the processing module determines to archive data when the metadata indicates that the user data comprises new banking records. As another example, the processing module determines to not archive data when the metadata indicates that the user data comprises routine website access information. The method repeats back to step **276** when the processing module determines not to archive data. The method continues to step **284** when the processing module determines to archive data.

(42) The method continues at step **284** where the processing module processes the user data to produce a data representation, wherein the data representation may be in a compressed and/or a transformed form to facilitate storage in a dispersed storage network (DSN) memory. The processing module processes the data based on one or more of the captured data, the metadata, the preferences, a processing method table lookup, a command, a message, and a predetermination. For example, the processing module processes the user data to produce a data representation where a size of the data representation facilitates an optimization of DSN memory storage efficiency. For instance, the data representation size may be determined to align with a data segment and data slice sizes such that memory is not unnecessarily underutilized as data blocks are stored in dispersed storage (DS) units of the DSN memory.

(43) The method continues at step **286** where the processing module determines operational parameters. Such a determination may be based on one or more of the data representation, the captured user data, the metadata, the preferences, a processing method table lookup, a command, a message, and a predetermination. For example, the processing module determines a pillar width and decode threshold such that an above average reliability approach to storing the data representation is provided when the processing module determines that the metadata indicates that the user data comprises very high priority financial records requiring a very long term of storage without failure.

(44) The method continues at step **288** where the processing module facilitates storage of the data representation in the DSN memory. For example, the processing module dispersed storage error encodes the data representation utilizing the operational parameters to produce encoded data slices. Next, the processing module sends the encoded data slices to the DS units of the DSN memory for storage therein.

(45) FIG. **11** is a flowchart illustrating an example of generating integrity information. The method begins at step **102** where a processing module receives a store data object message. Such a store data object message may include one or more of data, a user identifier (ID), a request, a data ID, a data object name, a data object, a data type indicator, a data object hash, a vault ID, a data size indicator, a priority indicator, a security indicator, and a performance indicator. The method continues at step **104** where the processing module determines dispersed storage error coding parameters (e.g., operational parameters) including one or more of a pillar width, a write threshold, a read threshold, an encoding method, a decoding method, an encryption method, a decryption method, a key, a secret key, a public key, a private key, a key reference, and an integrity information generation method designator. Such a determination may be based on one or more of information received in the store data object message, the user ID, the data ID, a vault lookup, a list, a command, a message, and a predetermination.

(46) The method continues at step **106** where the processing module dispersed storage error encodes data to produce a plurality of sets of encoded data slices in accordance with the dispersed storage error coding parameters. In addition, the processing module determines a plurality of sets of slice names corresponding to the plurality of sets of encoded data slices; where a slice name includes one or more of a slice index, a vault ID, a generation, an object number, and a segment number. Within a slice name, the slice index indicates a pillar number of a pillar width associated with the dispersed storage error coding parameters, the vault ID indicates a storage resource of a storage system common to one or more user devices, the generation indicates portions of a corresponding vault, the object number is associated with the data ID (e.g., a hash of the data ID), and the segment number indicates a segment identifier associated with one of a plurality of data segments (e.g., the plurality of data segment constitutes the data, a data file, etc.).

(47) The method continues at step **108** where the processing module determines integrity information for the plurality of sets of slice names. Such a determination may be in accordance with one or more integrity methods. In a first integrity method, the processing module generates individual integrity information for at least some of the slice names of at least some of the plurality of sets of slice names (e.g., at a slice name level) and generates the integrity information based on the individual integrity information. The individual integrity information may be generated by performing one or more of a hash function, cyclic redundancy check, encryption function, an encrypted digital signature function (e.g., digital signature algorithm (DSA), El Gamal, Elliptic Curve DSA, Rivest, Shamir and Adleman (RSA)), and parity check on a slice name of the at least some of the slices names of at least some of the plurality of sets of slices names to generate the individual integrity information. The hash function may include a hashed message authentication code (e.g., secure hash algorithm 1 (SHA1), hashed message authentication code message digest algorithm 5 (HMAC-MD5)) that uses a shared key and the encryption function includes an encryption algorithm that utilizes a private key, which is paired to a public key. In an example of generating individual integrity information, the processing module calculates a hash of at least some of the slice names and then encrypts the hash in accordance with an encryption method to produce an encrypted digital signature.

(48) In a second integrity method, the processing module generates set integrity information for a set of slice names of at least some of the plurality of sets of slice names (e.g., at a set level) and generates the integrity information based on the set integrity information. The set integrity information may be generated by performing one or more of the hash function, the cyclic redundancy check, the encryption function, the encrypted digital signature function, and the parity check on the set of slice names of at least some of the plurality of sets of slice names to generate the set integrity information.

(49) In a third integrity method, the processing module generates pillar integrity information for a pillar set of slice names of at least some of the plurality of sets of slice names (e.g., at a pillar level) and generates the integrity information based on the pillar integrity information. The pillar integrity

information may be generated by performing one or more of the hash function, the cyclic redundancy check, the encryption function, the encrypted digital signature function, and the parity check on the pillar set of slice names of at least some of the plurality of sets of slice names to generate the pillar integrity information.

(50) In a fourth integrity method, the processing module generates data file integrity information for at least some of the plurality of sets of slice names (e.g., at the data file level) and generates the integrity information based on the data file integrity information. The data file integrity information may be generated by performing one or more of the hash function, the cyclic redundancy check, the encryption function, the encrypted digital signature function, and the parity check on the at least some of the plurality of sets of slice names to generate the data file integrity information.

(51) In a fifth integrity method, the processing module generates combined integrity information for at least some of the encoded data slices of the plurality of sets of encoded data slices and for at least some of the slices names of at least some of the plurality of sets of slice names and generates the integrity information based on the combined integrity information. The combined integrity information includes performing one or more of the hash function, the cyclic redundancy check, the encryption function, the encrypted digital signature function, and the parity check on two or more of an encoded data slice of the at least some of the encoded data slices of the plurality of encoded data slices, a revision identifier, and an associated slice name of the at least some of the slice names of the plurality of sets of slice names to generate the combined integrity information. For example, the processing module performs an RSA encrypted digital signature on a combination of an encoded data slice and an associated slice name to generate the combined integrity information. As another example, the processing module performs a HMAC function on a set of combinations of encoded data slices and associated slice names name to generate the combined integrity information. As yet another example, the processing module performs a DSA encrypted digital signature on a combination of an encoded data slice, an associated slice name, and an associated revision identifier to generate the combined integrity information.

(52) The integrity information may be generated as a combination of the various methods. For example, the processing module performs the first integrity method, the fourth integrity method, and at least one of the second and third integrity methods to generate the integrity information.

(53) The method continues at step **110** where the processing module appends the integrity information to the slice name information to produce appended slice name information. For example, the processing module appends a HMAC digest to the slice name, revision, and date of a single encoded data slice. The method continues at step **112** where the processing module determines a dispersed storage (DS) unit storage set. Such a determination may be based on one or more of information received in the store data object message, a vault lookup, a list, a command, a message, a predetermination, the dispersed storage error coding parameters, encoded data slices, a dispersed storage network (DSN) memory status indicator, the slice name information, a virtual DSN address to physical location table lookup, and the integrity information. The method continues at step **114** where the processing module sends the plurality of sets of encoded data slices, the plurality of sets of slice names, and the integrity information to a DSN memory for storage therein.

(54) FIG. **12** is a flowchart illustrating an example of verifying slice integrity, which includes similar steps to FIG. **11**. The method begins with step **116** where a processing module receives a data retrieval request. Such a data retrieval request includes one or more of a retrieve data object request, a user identifier (ID), a data object name, a data ID, a data type indicator, a data object hash, a vault ID, a data size indicator, a priority indicator, a security indicator, and a performance indicator. The method continues with step **104** of FIG. **11** where the processing module determines dispersed storage error coding parameters (e.g., operational parameters) and with step **112** of FIG. **11** where the processing module determines a dispersed storage (DS) unit storage set.

(55) The method continues at step **122** where the processing module determines a plurality of sets

of slice names in accordance with the data retrieval request. Such a determination may be based on one or more of the data ID, the user ID, the vault ID, the dispersed storage error coding parameters, and extraction of a data size indicator from a reproduced data segment. The method continues at step **124** where the processing module receives stored integrity information corresponding to the data retrieval request. For example, the processing module sends one or more stored integrity information request messages to the DS unit storage set and receives the stored integrity information in response, wherein the stored integrity information request messages include at least some of the plurality of sets of slice names. Note that the stored integrity information and associated encoded data slices were previously stored in the DS unit storage set.

(56) The method continues at step **126** where the processing module generates desired integrity information based on the plurality of sets of slice names. Such a generation of the desired integrity information may be based on one or more of the five integrity methods discussed with reference to FIG. **11**.

(57) The method continues at step **128** where the processing module receives encoded data slices. For example, the processing module sends encoded data slice retrieval messages to the DS unit storage set that includes at least some of the plurality of sets of slice names and receives the encoded data slices in response. The method continues at step **130** where the processing module compares the stored integrity information with the desired integrity information. For example, the processing module decrypts the stored integrity information using a public key of a public-private key pair to produce at least some of a plurality of sets of reconstructed slice names and compares corresponding slices names of the plurality of sets of slice names with the at least some of a plurality of sets of reconstructed slice names when the stored integrity information includes encrypted slice names (e.g., a digital signature, encrypted slice names) encrypted with a private key of the super public-private key pair.

(58) As another example of comparing at step **130**, the processing module decrypts the stored integrity information using a shared key to produce the at least some of the plurality of sets of reconstructed slice names a compares corresponding slice names of the plurality of sets of slice names with the at least some of the plurality of sets of reconstructed slice names when the stored integrity information includes encrypted slice names encrypted with the shared key. The processing module compares the stored integrity information directly to the desired integrity from a when the stored integrity information includes results of a hash function (e.g., a HMAC).

(59) As yet another example of comparing at step **130**, when multiple integrity methods are used (e.g., the fourth integrity and the second or third integrity methods), a higher level comparison is performed first (e.g., use the forth integrity method). If it successful, the method continues based on a favorable comparison. If, however, the higher level comparison was not successful, a lower level comparison is performed (e.g., the second or third integrity method). For sets of encoded data slices that the lower level comparison was successful, the method continues based on a favorable comparison. For the sets of encoded data slices that the lower level comparison was not successful, an error is generated or another comparison may be performed based on the individual integrity information.

(60) As a specific example, an integrity check is initially performed for the data file. If successful, the method continues with decoding the plurality of sets of encoded data slices to recapture the data file (e.g., step **134**). If the data file level integrity check was not successful, then a set level integrity check is performed to identify which sets have an error. For each set having an error, an individual encoded slice name integrity check may be performed to identify encoded data slices having an error. The encoded data slices having an error or sets including slices having an error may be excluded from the decoding (e.g., step **132**). Even if encoded data slices have an error, as long as a decode threshold number of encoded data slices per set are available, then the data file can be accurately reproduced.

(61) It is noted that terminologies as may be used herein such as bit stream, stream, signal

sequence, etc. (or their equivalents) have been used interchangeably to describe digital information whose content corresponds to any of a number of desired types (e.g., data, video, speech, text, graphics, audio, etc. any of which may generally be referred to as 'data').

(62) As may be used herein, the terms "substantially" and "approximately" provides an industry-accepted tolerance for its corresponding term and/or relativity between items. For some industries, an industry-accepted tolerance is less than one percent and, for other industries, the industry-accepted tolerance is 10 percent or more. Other examples of industry-accepted tolerance range from less than one percent to fifty percent. Industry-accepted tolerances correspond to, but are not limited to, component values, integrated circuit process variations, temperature variations, rise and fall times, thermal noise, dimensions, signaling errors, dropped packets, temperatures, pressures, material compositions, and/or performance metrics. Within an industry, tolerance variances of accepted tolerances may be more or less than a percentage level (e.g., dimension tolerance of less than +/−1%). Some relativity between items may range from a difference of less than a percentage level to a few percent. Other relativity between items may range from a difference of a few percent to magnitude of differences.

(63) As may also be used herein, the term(s) "configured to", "operably coupled to", "coupled to", and/or "coupling" includes direct coupling between items and/or indirect coupling between items via an intervening item (e.g., an item includes, but is not limited to, a component, an element, a circuit, and/or a module) where, for an example of indirect coupling, the intervening item does not modify the information of a signal but may adjust its current level, voltage level, and/or power level. As may further be used herein, inferred coupling (i.e., where one element is coupled to another element by inference) includes direct and indirect coupling between two items in the same manner as "coupled to".

(64) As may even further be used herein, the term "configured to", "operable to", "coupled to", or "operably coupled to" indicates that an item includes one or more of power connections, input(s), output(s), etc., to perform, when activated, one or more its corresponding functions and may further include inferred coupling to one or more other items. As may still further be used herein, the term "associated with", includes direct and/or indirect coupling of separate items and/or one item being embedded within another item.

(65) As may be used herein, the term "compares favorably", indicates that a comparison between two or more items, signals, etc., provides a desired relationship. For example, when the desired relationship is that signal **1** has a greater magnitude than signal **2**, a favorable comparison may be achieved when the magnitude of signal **1** is greater than that of signal **2** or when the magnitude of signal **2** is less than that of signal **1**. As may be used herein, the term "compares unfavorably", indicates that a comparison between two or more items, signals, etc., fails to provide the desired relationship.

(66) As may be used herein, one or more claims may include, in a specific form of this generic form, the phrase "at least one of a, b, and c" or of this generic form "at least one of a, b, or c", with more or less elements than "a", "b", and "c". In either phrasing, the phrases are to be interpreted identically. In particular, "at least one of a, b, and c" is equivalent to "at least one of a, b, or c" and shall mean a, b, and/or c. As an example, it means: "a" only, "b" only, "c" only, "a" and "b", "a" and "c", "b" and "c" and/or "a" "b", and "c".

(67) As may also be used herein, the terms "processing module", "processing circuit", "processor", "processing circuitry", and/or "processing unit" may be a single processing device or a plurality of processing devices. Such a processing device may be a microprocessor, micro-controller, digital signal processor, microcomputer, central processing unit, field programmable gate array, programmable logic device, state machine, logic circuitry, analog circuitry, digital circuitry, and/or any device that manipulates signals (analog and/or digital) based on hard coding of the circuitry and/or operational instructions. The processing module, module, processing circuit, processing circuitry, and/or processing unit may be, or further include, memory and/or an integrated memory

element, which may be a single memory device, a plurality of memory devices, and/or embedded circuitry of another processing module, module, processing circuit, processing circuitry, and/or processing unit. Such a memory device may be a read-only memory, random access memory, volatile memory, non-volatile memory, static memory, dynamic memory, flash memory, cache memory, and/or any device that stores digital information. Note that if the processing module, module, processing circuit, processing circuitry, and/or processing unit includes more than one processing device, the processing devices may be centrally located (e.g., directly coupled together via a wired and/or wireless bus structure) or may be distributedly located (e.g., cloud computing via indirect coupling via a local area network and/or a wide area network). Further note that if the processing module, module, processing circuit, processing circuitry and/or processing unit implements one or more of its functions via a state machine, analog circuitry, digital circuitry, and/or logic circuitry, the memory and/or memory element storing the corresponding operational instructions may be embedded within, or external to, the circuitry comprising the state machine, analog circuitry, digital circuitry, and/or logic circuitry. Still further note that, the memory element may store, and the processing module, module, processing circuit, processing circuitry and/or processing unit executes, hard coded and/or operational instructions corresponding to at least some of the steps and/or functions illustrated in one or more of the Figures. Such a memory device or memory element can be included in an article of manufacture.

(68) One or more embodiments have been described above with the aid of method steps illustrating the performance of specified functions and relationships thereof. The boundaries and sequence of these functional building blocks and method steps have been arbitrarily defined herein for convenience of description. Alternate boundaries and sequences can be defined so long as the specified functions and relationships are appropriately performed. Any such alternate boundaries or sequences are thus within the scope and spirit of the claims. Further, the boundaries of these functional building blocks have been arbitrarily defined for convenience of description. Alternate boundaries could be defined as long as the certain significant functions are appropriately performed. Similarly, flow diagram blocks may also have been arbitrarily defined herein to illustrate certain significant functionality.

(69) To the extent used, the flow diagram block boundaries and sequence could have been defined otherwise and still perform the certain significant functionality. Such alternate definitions of both functional building blocks and flow diagram blocks and sequences are thus within the scope and spirit of the claims. One of average skill in the art will also recognize that the functional building blocks, and other illustrative blocks, modules and components herein, can be implemented as illustrated or by discrete components, application specific integrated circuits, processors executing appropriate software and the like or any combination thereof.

(70) In addition, a flow diagram may include a “start” and/or “continue” indication. The “start” and “continue” indications reflect that the steps presented can optionally be incorporated in or otherwise used in conjunction with one or more other routines. In addition, a flow diagram may include an “end” and/or “continue” indication. The “end” and/or “continue” indications reflect that the steps presented can end as described and shown or optionally be incorporated in or otherwise used in conjunction with one or more other routines. In this context, “start” indicates the beginning of the first step presented and may be preceded by other activities not specifically shown. Further, the “continue” indication reflects that the steps presented may be performed multiple times and/or may be succeeded by other activities not specifically shown. Further, while a flow diagram indicates a particular ordering of steps, other orderings are likewise possible provided that the principles of causality are maintained.

(71) The one or more embodiments are used herein to illustrate one or more aspects, one or more features, one or more concepts, and/or one or more examples. A physical embodiment of an apparatus, an article of manufacture, a machine, and/or of a process may include one or more of the aspects, features, concepts, examples, etc. described with reference to one or more of the

embodiments discussed herein. Further, from figure to figure, the embodiments may incorporate the same or similarly named functions, steps, modules, etc. that may use the same or different reference numbers and, as such, the functions, steps, modules, etc. may be the same or similar functions, steps, modules, etc. or different ones.

(72) Unless specifically stated to the contra, signals to, from, and/or between elements in a figure of any of the figures presented herein may be analog or digital, continuous time or discrete time, and single-ended or differential. For instance, if a signal path is shown as a single-ended path, it also represents a differential signal path. Similarly, if a signal path is shown as a differential path, it also represents a single-ended signal path. While one or more particular architectures are described herein, other architectures can likewise be implemented that use one or more data buses not expressly shown, direct connectivity between elements, and/or indirect coupling between other elements as recognized by one of average skill in the art.

(73) The term “module” is used in the description of one or more of the embodiments. A module implements one or more functions via a device such as a processor or other processing device or other hardware that may include or operate in association with a memory that stores operational instructions. A module may operate independently and/or in conjunction with software and/or firmware. As also used herein, a module may contain one or more sub-modules, each of which may be one or more modules.

(74) As may further be used herein, a computer readable memory includes one or more memory elements. A memory element may be a separate memory device, multiple memory devices, or a set of memory locations within a memory device. Such a memory device may be a read-only memory, random access memory, volatile memory, non-volatile memory, static memory, dynamic memory, flash memory, cache memory, and/or any device that stores digital information. The memory device may be in a form a solid-state memory, a hard drive memory, cloud memory, thumb drive, server memory, computing device memory, and/or other physical medium for storing digital information.

(75) While particular combinations of various functions and features of the one or more embodiments have been expressly described herein, other combinations of these features and functions are likewise possible. The present disclosure is not limited by the particular examples disclosed herein and expressly incorporates these other combinations.

Claims

1. A method comprising: storing a plurality of data in a storage system; determining a plurality of identifiers corresponding to the plurality of data; generating, for the plurality of data, a first set of integrity information corresponding to a first system storage level by performing a first set of cyclic redundancy checks; generating, for the plurality of data, a plurality of corresponding data structures that each include: a corresponding one of the plurality of identifiers; and corresponding integrity information of the first set of integrity information; storing the first set of integrity information and the plurality of identifiers in the storage system via storage of the plurality of corresponding data structures; generating, for the plurality of data, a second set of integrity information corresponding to a second system storage level by performing a second set of cyclic redundancy checks; and storing the second set of integrity information in the storage system.

2. The method of claim 1, wherein the plurality of data is generated based on erasure coding.

3. The method of claim 1, wherein the plurality of identifiers are associated with at least one data slice generated via an encoding process in accordance with a width, and wherein a corresponding decoding process can accommodate a number of failures equal to the width minus an error coding parameter of the encoding process.

4. The method of claim 1, wherein the plurality of identifiers identify a virtual memory space that maps to storage units of the storage system.

5. The method of claim 1, wherein the plurality of identifiers are determined in conjunction with determining a plurality of virtual memory addresses.
6. The method of claim 5, wherein each virtual memory address of the plurality of virtual memory addresses is associated with a physical address, and wherein the integrity information is generated based on the plurality of virtual memory addresses.
7. The method of claim 1, further comprising: performing periodic data storage integrity verification based on accessing at least one of the second set of integrity information in the storage system; wherein performing the periodic data storage integrity verification is based on periodically determining whether any of the plurality of data has been corrupted.
8. The method of claim 7, further comprising: rebuilding at least some of the plurality of data based on determining the at least some of the plurality of data has been corrupted.
9. The method of claim 1, wherein the storage system is configured to store the data via a plurality of system storage levels that includes the first system storage level and the second system storage level.
10. The method of claim 1, wherein the first set of integrity information is generated based on a first plurality of data portions of the plurality of data having first data structuring corresponding to the first system storage level, and wherein the second set of integrity information is generated based on a second plurality of data portions of the plurality of data having second data structuring corresponding to the second system storage level.
11. The method of claim 10, wherein each of the first plurality of data portions include multiple ones of the second plurality of data portions.
12. A computer comprises: a memory; and a processing module operable to: store a plurality of data in a storage system; determine a plurality of identifiers corresponding to the plurality of data; generate, for the plurality of data, a first set of integrity information corresponding to a first system storage level by performing a first set of cyclic redundancy checks; generate, for the plurality of data, a plurality of corresponding data structures that each include: a corresponding one of the plurality of identifiers; and corresponding integrity information of the first set of integrity information; store the first set of integrity information and the plurality of identifiers in the storage system via storage of the plurality of corresponding data structures; generate, for the plurality of data, a second set of integrity information corresponding to a second system storage level by performing a second set of cyclic redundancy checks; and store the second set of integrity information in the storage system.
13. The computer of claim 12, wherein the plurality of identifiers are associated with at least one data slice generated via an encoding process in accordance with a width, and wherein a corresponding decoding process can accommodate a number of failures equal to the width minus an error coding parameter of the encoding process.
14. The computer of claim 12, wherein the plurality of identifiers identify a virtual memory space that maps to storage units of the storage system.
15. The computer of claim 12, wherein the plurality of identifiers are determined in conjunction with determining a plurality of virtual memory addresses.
16. The computer of claim 15, wherein each virtual memory address of the plurality of virtual memory addresses is associated with a physical address, and wherein the integrity information is generated based on the plurality of virtual memory addresses.
17. The computer of claim 12, further comprising: performing periodic data storage integrity verification based on accessing at least one of the second set of integrity information in the storage system; wherein performing the periodic data storage integrity verification is based on periodically determining whether any of the plurality of data has been corrupted.
18. The computer of claim 17, further comprising: rebuilding at least some of the plurality of data based on determining the at least some of the plurality of data has been corrupted.
19. A storage system comprises: a plurality of storage units; and at least one processing module

operable to: store a plurality of data; determine a plurality of identifiers corresponding to the plurality of data; generate, for the plurality of data, a first set of integrity information corresponding to a first system storage level by performing a first set of cyclic redundancy checks; generate, for the plurality of data, a plurality of corresponding data structures that each include: a corresponding one of the plurality of identifiers; and corresponding integrity information of the first set of integrity information; store the first set of integrity information and the plurality of identifiers in the storage system via storage of the plurality of corresponding data structures; generate, for the plurality of data, a second set of integrity information corresponding to a second system storage level by performing a second set of cyclic redundancy checks; and store the second set of integrity information in the storage system.

20. The storage system of claim 19, wherein the plurality of data is generated based on erasure coding.